



UPLA
UNIVERSIDAD PERUANA
LOS ANDES

*Formando líderes para
transformar el futuro*



SEMANA 13:

SQL



DESARROLLO INFOGRAFÍAS DE APRENDIZAJE



CURSO:

BASE DE DATOS II



DOCENTE:

ING. FERNANDEZ RAUL ENRIQUE



ESTUDIANTE:

**ROSALES CRISPIN
ARISTOTELES O.**



CICLO:

V



CARRERA:

**ING. SISTEMA Y
COMPUTACION**



**OPTIMIZAR HOY,
MEJORAR SIEMPRE
LOS PROCESOS DE
GRADOS Y TÍTULOS**



ANÁLISIS DE RENDIMIENTO CON SQL PROFILER Y EXTENDED EVENTS

APLICADO A LA BASE DE DATOS: **GradosTítulos**



OBJETIVO: Optimizar el rendimiento de la base de datos GradosTítulos mediante el monitoreo del comportamiento del motor SQL Server, el análisis de consultas, la optimización de índices, el control de transacciones y la administración de recursos del servidor, garantizando tiempos de respuesta adecuados para los procesos de grados y títulos.

1. ¿QUÉ ES?

SQL

SQL PROFILER

Herramienta clásica que permite capturar eventos, consultas y datos de rendimiento en tiempo real en SQL Server.

XE

EXTENDED EVENTS

Tecnología moderna y ligera para monitoreo detallado y escalable, diseñada para producción.



Ambas herramientas nos ayudan a entender qué sucede dentro del motor de SQL Server al trabajar con la base **GradosTítulos**.

2. ARQUITECTURA

APLICACIONES
(Tramites, Pagos,
Sustentaciones,
Reportes, etc.)



SQL SERVER ENGINE



DESTINOS
DE DATOS



CAPTURA DE EVENTOS

SQL PROFILER
(Trace)

EXTENDED EVENTS
(Sesión XE)

3. FILTRADO DE EVENTOS

El filtrado es clave para capturar solo lo que importa en **GradosTítulos**.

Ejemplos de filtros recomendados:

- ✓ Por Base de Datos: GradosTítulos
- ✓ Por Usuario Aplicación
- ✓ Por Tipo de Evento (SQL Batch, SP:StmtCompleted, RPC:Completed)
- ✓ Por Duración (ej. > 1000 ms)
- ✓ Por Error (Severity >= 11)
- ✓ Por Objeto (Tablas críticas: Tramite, Sustentacion, TrabajoInvestigacion, Pago, Diploma)



4. ENTORNOS RECOMENDADOS



SQL PROFILER

- Uso en desarrollo y pruebas
- Análisis puntual y sesiones cortas
- No recomendado en producción por su alto consumo.



EXTENDED EVENTS

- Uso en desarrollo y producción
- Bajo impacto en el servidor
- Sesiones persistentes y escalables
- El estándar moderno de SQL Server

5. DESTINO DE DATOS

¿Dónde se almacenan los eventos capturados?



Archivo .xel
Ligero y eficiente. Ideal para análisis posterior en SSMS.



Tabla (ring_buffer)
En memoria. Rápido, pero con capacidad limitada.



Evento en Vivo (Live Data)
Visualización en tiempo real durante el monitoreo.



Event File (sincrónico/asíncrono)
Para cargas pesadas y sesiones prolongadas.

6. DIAGNÓSTICOS RÁPIDOS CON SQL PROFILER / XE



Consultas lentas
Identifica instrucciones que tardan más de lo esperado.



Bloqueos y esperas
Detecta bloqueos, waits y procesos que afectan el rendimiento.



Errores frecuentes
Captura errores de ejecución que impactan los procesos.



Uso de procedimientos
Analiza SP_RegistrarTramite, SP_RegistrarSustentacion y otros.

Ejemplo de eventos útiles:

- ✓ SQL:BatchCompleted
- ✓ RPC:Completed
- ✓ SP:StmtCompleted
- ✓ Attention
- ✓ Deadlock Graph
- ✓ Lock:Deadlock
- ✓ Error Reported
- ✓ Showplan XML



7. EXTENDED EVENTS: EL ESTÁNDAR PARA PRODUCCIÓN



Menor consumo de CPU y memoria



Más seguro y estable



Sesiones persistentes



Mayor control sobre los eventos



Mejor integración con SQL Server Management Studio

Ejemplo de sesión XE para **GradosTítulos**

Eventos recomendados:

- sql_statement_completed
- rpc_completed
- lock_deadlock
- error_reported
- wait_info

Filtros sugeridos:

- database_id = GradosTítulos
- duration > 1000
- sqlserver.session_id <> 0

XE

8. ESTRATEGIA DE MITIGACIÓN TRAS EL ANÁLISIS



1. ANALIZAR

Identificar consultas lentas, bloqueos, errores y eventos críticos.



2. OPTIMIZAR

- Crear/ajustar índices
- Actualizar estadísticas
- Optimizar consultas y SP.



3. CONTROLAR TRANSACCIONES

Reducir bloqueos, usar índices adecuados y aislar transacciones.



4. MONITOREAR CONTINUAMENTE

Usar Extended Events como sistema de monitoreo permanente.



5. VALIDAR

Verificar mejoras en tiempos de respuesta y estabilidad del sistema.

Beneficio para **GradosTítulos**



Procesos más rápidos, reportes eficientes y mejor experiencia para estudiantes y personal administrativo.



APLICACIÓN DIRECTA EN GRADOS TITULOS

Úsalo para monitorear trámites, pagos, sustentaciones, generación de diplomas, reportes académicos y todos los procesos críticos de la universidad.



MONITOREA, ANALIZA,
OPTIMIZA Y ASEGURA
EL RENDIMIENTO.



RENDIMIENTO ÓPTIMO
MEJORES TIEMPOS DE RESPUESTA
PROCESOS ACADÉMICOS EFICIENTES

ESTADÍSTICAS E ÍNDICES

(CREACIÓN, FRAGMENTACIÓN, MANTENIMIENTO)

APLICADO A LA BASE DE DATOS: **GradosTitulos**

OBJETIVO: Mantener el optimizador de SQL Server informado y acelerar las consultas de la base de datos **GradosTitulos** mediante el uso correcto de estadísticas e índices, su diagnóstico y mantenimiento.

1. ¿QUÉ ES?



ESTADÍSTICAS

Son resúmenes de la distribución de datos en las columnas. Permiten al optimizador elegir el mejor plan de ejecución.



ÍNDICES

Son estructuras que aceleran la búsqueda y el ordenamiento de datos, reduciendo lecturas en disco y mejorando el rendimiento.



Sin estadísticas precisas ni índices adecuados, las consultas son lentas e ineficientes.

2. ESTADÍSTICAS: EL MAPA DEL OPTIMIZADOR



Indican cuántos valores distintos existen.



Muestran la densidad y distribución de datos.



Ayudan a estimar filas y elegir el mejor plan.



Se crean automáticamente, pero deben mantenerse actualizadas.

COMANDOS ÚTILES

```
-- Actualizar estadísticas
UPDATE STATISTICS
Tramite.Tramite;

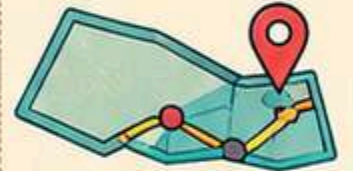
-- Estadísticas de una tabla
DBCC SHOW_STATISTICS ('Tramite.Tramite');

-- Crear estadísticas en columna específica
CREATE STATISTICS ST_EstadoTramite
ON Tramite.Tramite (IdEstadoTramite);
```

EJEMPLO EN **GradosTitulos**

Columnas críticas:

- ✓ Tramite(IdEstadoTramite)
- ✓ Persona(DNI)
- ✓ Sustentacion(IdTramite)
- ✓ Pago(IdTramite)
- ✓ TrabajoInvestigacion(IdLinea)



3. ÍNDICES: CREACIÓN ESTRATÉGICA

TIPOS PRINCIPALES



ÍNDICE CLÚSTER (CLUSTERED)

Define el orden físico de los datos. Solo puede existir uno por tabla.



ÍNDICE NO CLÚSTER (NON-CLUSTERED)

Estructura separada que apunta a los datos. Pueden existir varios.



ÍNDICE ÚNICO (UNIQUE)

No permite valores duplicados.



ÍNDICE INCLUIDO (INCLUDE) COVERING INDEX

Incluye columnas adicionales para cubrir la consulta sin ir a la tabla base.

SINTAXIS BÁSICA

```
-- CLÚSTER (Clustered)
CREATE CLUSTERED INDEX IX_Matricula_PK
ON Academico.Matricula (IdMatricula);
```



```
-- NO CLÚSTER (Non-Clustered)
CREATE NONCLUSTERED INDEX IX_Tramite_Estado
ON Tramite.Tramite (IdEstadoTramite)
INCLUDE (FechaRegistro, IdTipoTramite);
```



```
-- ÚNICO (Unique)
CREATE UNIQUE INDEX IX_Persona_DNI
ON Academico.Persona (DNI);
```



ÍNDICES RECOMENDADOS EN **GradosTitulos**

- ✓ Tramite(IdEstadoTramite, FechaRegistro)
- ✓ TrabajoInvestigacion(IdTramite)
- ✓ Sustentacion(IdTramite)
- ✓ Pago(IdTramite, FechaPago)
- ✓ Persona(DNI)
- ✓ Matricula(IdPersona, IdPrograma)



Construir índices según los patrones de consulta más frecuentes evita lecturas innecesarias.

4. FRAGMENTACIÓN: DIAGNÓSTICO CRÍTICO

La fragmentación ocurre cuando los datos se almacenan en páginas no contiguas, degradando el rendimiento de los índices.

NIVELES DE FRAGMENTACIÓN (%)

- 0 - 5 Óptimo
- 5 - 30 Aceptable
- 30 - 80 Alto (Reorganizar)
- > 80 Muy Alto (Reconstruir)

CONSULTAR FRAGMENTACIÓN

```
SELECT
OBJECT_NAME(ps.object_id) AS Tabla,
i.name AS Indice,
ps.avg_fragmentation_in_percent,
ps.page_count
FROM sys.dm_db_index_physical_stats
(OB_ID(), NULL, NULL, NULL, 'LIMITED') ps
JOIN sys.indexes i
ON ps.object_id = i.object_id
AND ps.index_id = i.index_id
WHERE i.name IS NOT NULL
ORDER BY ps.avg_fragmentation_in_percent DESC;
```



5. ESTRATEGIA DE MANTENIMIENTO

¿QUÉ HACER?

FRAGMENTACIÓN	ACCIÓN RECOMENDADA
0 - 5%	No hacer nada
5 - 30%	REORGANIZE INDEX (sin bloqueo fuerte)
30 - 80%	REORGANIZE INDEX
> 80%	REBUILD INDEX (reconstrucción completa)

COMANDOS

```
-- Reorganizar
ALTER INDEX IX_Tramite_Estado
ON Tramite.Tramite REORGANIZE;

-- Reconstruir
ALTER INDEX IX_Tramite_Estado
ON Tramite.Tramite REBUILD;
```



Ejecutar mantenimiento regularmente mejora el rendimiento y reduce el consumo de recursos.

6. AUTOMATIZACIÓN PROFESIONAL

USAR SQL SERVER AGENT



- Crear Job de mantenimiento.
- Definir frecuencia (diario / semanal).
- Ejecutar scripts de mantenimiento.
- Registrar historial y alertas.



La automatización garantiza consistencia, disponibilidad y rendimiento continuo en **GradosTitulos**.

EJEMPLO DE JOB (T-SQL)

```
EXEC sp_MSforeachtable
'ALTER INDEX ALL ON ?
REORGANIZE WITH (LOB_COMPACTION = ON);';

-- Para reconstrucción
EXEC sp_MSforeachtable
'ALTER INDEX ALL ON ?
REBUILD WITH (ONLINE = ON);';
```



7. BENEFICIOS EN **GradosTitulos**



Consultas más rápidas en trámites, pagos, sustentaciones y reportes.



Menor consumo de CPU, memoria y lecturas de disco.



Planes de ejecución más eficientes y estables.



Mayor disponibilidad para los procesos académicos y administrativos.



REGLA DE ORO

Estadísticas actualizadas + Índices adecuados + Mantenimiento constante
= Rendimiento óptimo y usuarios satisfechos.



CURSO:
BASE DE DATOS

ADMINISTRACIÓN DE TRANSACCIONES Y BLOQUEOS

APLICADO A LA BASE DE DATOS: **GradosTítulos**

OBJETIVO: Garantizar la integridad, consistencia y disponibilidad de los datos en los procesos de **Grados y Títulos** mediante el control adecuado de transacciones, bloqueos y niveles de aislamiento.

1. ¿QUÉ ES?



Las transacciones son unidades de trabajo que agrupan operaciones lógicas en la base de datos. Los bloqueos (locks) controlan el acceso concurrente a los datos, evitando inconsistencias.

EJEMPLOS EN GradosTítulos

- ✓ Registrar trámite y sus pagos.
- ✓ Sustentación de tesis.
- ✓ Emisión de resolución y diploma.
- ✓ Actualización de estados.



2. ADMINISTRACIÓN DE TRANSACCIONES (ACID)

A

Atomicidad
Todo o nada. Si falla una parte, se revierte toda la transacción.

C

Consistencia
Mantiene las reglas y restricciones de la base de datos.

I

Aislamiento
Las transacciones concurrentes no se interfieren entre sí.

D

Durabilidad
Los cambios confirmados persisten ante fallos.

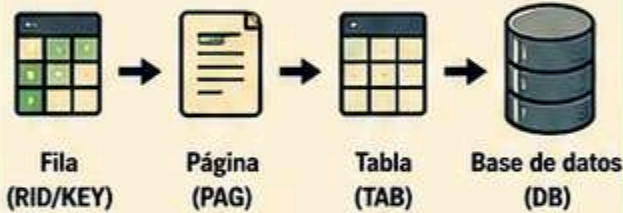
EJEMPLO EN GradosTítulos (Registrar pago y actualizar trámite)

```
BEGIN TRANSACTION;  
BEGIN TRY  
    INSERT INTO Tramite.Pago  
    (IdTramite, Monto, FechaPago, FormaPago, Estado)  
    VALUES ( @IdTramite, @Monto, GETDATE(), 'TRANSFERENCIA', 'CONFIRMADO');  
    UPDATE Tramite.Tramite  
    SET IdEstadoTramite = 3, FechaUltMod = GETDATE()  
    WHERE IdTramite = @IdTramite;  
COMMIT TRANSACTION;  
END TRY  
BEGIN CATCH  
    ROLLBACK TRANSACTION;  
    THROW;  
END CATCH;
```



3. JERARQUÍA Y TIPOS DE BLOQUEOS (LOCKS)

Jerarquía de granularidad (de menor a mayor):



SQL Server escala los bloqueos según la necesidad para proteger los datos y mantener el rendimiento.

4. TIPOS DE BLOQUEOS MÁS COMUNES



Compartido (S - Shared)
Permite lectura. Varios pueden coexistir. No permite escritura.



Lectura permitida



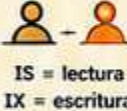
Exclusivo (X - Exclusive)
Permite lectura y escritura. Nadie más puede acceder mientras esté activo.



Lectura/ Escritura



De Intención (IS / IX)
Indica intención de tomar bloqueos más granulares. No bloquea acceso directo.



IS = lectura
IX = escritura

5. NIVELES DE AISLAMIENTO (ISOLATION LEVELS)

NIVEL	DESCRIPCIÓN	FENÓMENOS QUE EVITA
READ UNCOMMITTED	Permite lecturas sucias (mínimo aislamiento).	Ninguno
READ COMMITTED (Predeterminado)	No permite lecturas sucias.	Lecturas sucias
REPEATABLE READ	Evita lecturas sucias y lecturas no repetibles.	Lecturas sucias, lecturas no repetibles
SERIALIZABLE	Máximo aislamiento. Simula ejecución secuencial.	Lecturas sucias, lecturas no repetibles, phantom reads
SNAPSHOT	Usa versiones de fila (multi-versión). No bloquea lecturas.	Lecturas sucias, lecturas no repetibles

★ Para procesos de consultas masivas en GradosTítulos, se recomienda Snapshot o Read Committed Snapshot.

6. OPTIMIZACIÓN EN PRODUCCIÓN (RCSI)

Read Committed Snapshot Isolation (RCSI) permite lecturas consistentes sin bloquear ni ser bloqueado en el nivel **READ COMMITTED**. Ideal para sistemas de alta concurrencia como **GradosTítulos**.

HABILITAR RCSI

```
ALTER DATABASE GradosTítulos  
SET READ_COMMITTED_SNAPSHOT ON  
WITH ROLLBACK IMMEDIATE;
```



VERIFICAR ESTADO

```
SELECT name, is_read_committed_snapshot_on  
FROM sys.databases  
WHERE name = 'GradosTítulos';
```

BENEFICIOS

- ✓ Reduce bloqueos de lectura.
- ✓ Mejora la concurrencia.
- ✓ Mayor escalabilidad.
- ✓ No requiere cambios en las consultas.



CONSIDERACIONES

Aumenta el uso de tempdb (almacena versiones de fila). Monitorear su tamaño y rendimiento.

7. DIAGNÓSTICO DE BLOQUEOS EN TIEMPO REAL

SESIÓN BLOQUEADORA Y BLOQUEADA

```
SELECT  
    r.session_id AS Sesión,  
    r.status AS Estado,  
    r.blocking_session_id AS Bloqueador,  
    r.wait_type AS Espera,  
    r.wait_time AS Espera_ms,  
    r.wait_resource AS Recurso,  
    t.text AS Consulta  
FROM sys.dm_exec_requests r  
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t  
WHERE r.blocking_session_id > 0  
ORDER BY r.wait_time DESC;
```

DETALLE DE BLOQUEOS (LOCKS)

```
SELECT  
    tl.request_session_id AS Sesión,  
    tl.resource_type AS TipoRecurso,  
    tl.resource_description AS Recurso,  
    tl.request_mode AS Modo,  
    tl.request_status AS Estado  
FROM sys.dm_tran_locks tl  
ORDER BY tl.request_session_id;
```



CADENA DE BLOQUEO (CHAIN)

```
SELECT *  
FROM sys.dm_os_waiting_tasks  
WHERE wait_type LIKE 'LCK%'  
ORDER BY wait_duration_ms DESC;
```



Interpretación rápida:

- blocking_session_id > 0 => Hay bloqueo.
- wait_type = LCK_M_... => Esperando por un Lock.
- resource_type indica el nivel: KEY, PAGE, OBJECT, DATABASE.

8. ESTRATEGIA DE MITIGACIÓN

1. DISEÑO ÍNDICES ADECUADOS



Índices correctos reducen escaneos y la necesidad de bloqueos a nivel de tabla.

2. CONSULTAS EFICIENTES



Evitar SELECT *, subconsultas innecesarias y funciones en predicados.

3. TRANSACCIONES CORTAS



Hacer transacciones rápidas y enfocadas. Menos tiempo = menos bloqueo.

4. ELEGIR NIVEL DE AISLAMIENTO ADECUADO



Usar RCSI o Snapshot para consultas. Serializable solo cuando sea necesario.

5. MONITOREO PERMANENTE



Uso de DMVs, alertas y Query Store para detectar problemas antes que afecten a los usuarios.

TABLAS CRÍTICAS EN GradosTítulos (Ejemplos)



Tramite.Tramite
Alta concurrencia por cambios de estado.



Tramite.Pago
Registros frecuentes.



Evaluacion.Sustentacion
Actualizaciones por notas y estados.



Documento.Diploma
Generación y entrega.



BUENA PRÁCTICA

Transacciones bien administradas + Bloqueos controlados
= Integridad de datos + Rendimiento óptimo + Usuarios satisfechos



CURSO: BASE DE DATOS



ANÁLISIS DE PLANES DE EJECUCIÓN

APLICADO A LA BASE DE DATOS: **GradosTitulos**



OBJETIVO: Identificar cuellos de botella, consultas ineficientes y oportunidades de mejora en la base de datos **GradosTitulos** mediante el análisis de planes de ejecución del motor SQL Server.

1. ¿QUÉ ES?



El plan de ejecución es la estrategia que SQL Server usa para obtener los resultados de una consulta. Muestra el orden de las operaciones y los recursos estimados o reales que utilizará el motor.



Analizarlo correctamente permite mejorar el rendimiento de las consultas en **GradosTitulos**.

2. TIPOS DE PLANES DE EJECUCIÓN

PLAN DE EJECUCIÓN ESTIMADO (Sin ejecutar la consulta)



- Muestra estimaciones de costos, filas y uso de recursos.
- Útil para probar y comparar antes de ejecutar.



CÓMO OBTENERLO

CTRL + L o Incluir Plan Estimado (Show Estimated Execution Plan)

PLAN DE EJECUCIÓN REAL (Después de ejecutar la consulta)



- Muestra estadísticas reales: filas, lecturas, tiempo, CPU, etc.
- Fundamental para identificar problemas reales.

CÓMO OBTENERLO

CTRL + M o Incluir Plan Actual (Include Actual Execution Plan)

HERRAMIENTAS EN SSMS



Execution Plan (Gráfico)
Ctrl + L / Ctrl + M



Estadísticas de IO y Tiempo
Ctrl + Shift + S



Propiedades del Operador (F4)



Mostrar XML del Plan
(Click derecho > Showplan XML)

3. ANATOMÍA DE UN PLAN: OPERADORES FUNDAMENTALES



SELECT
Devuelve los resultados finales.



INDEX SEEK
Busca filas usando un índice. Muy eficiente.



INDEX SCAN
Recorre todo el índice. Menos eficiente que Seek.



TABLE SCAN
Lee toda la tabla. Costoso en tablas grandes.



NESTED LOOPS una.
Bueno en conjuntos pequeños.



HASH MATCH
Une grandes volúmenes de datos.



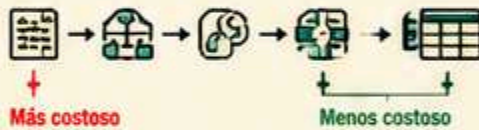
SORT
Ordena los datos. Puede ser costoso.



KEY LOOKUP (RID/KEY LOOKUP)
Busca columnas faltantes en el índice.

FLUJO DEL PLAN

El flujo va de derecha a izquierda: SQL Server comienza en el operador más a la derecha y termina en SELECT.



COSTO DEL PLAN

El costo del plan es relativo (no es tiempo real). El operador con mayor costo suele ser el cuello de botella.

- Alto costo (Problema potencial)
- Medio costo (Analizar)
- Bajo costo (Eficiente)



4. ALERTAS CRÍTICAS: ¿QUÉ BUSCAR EN EL PLAN?



TABLE SCAN
en tablas grandes

Indica que no se está usando un índice adecuado.



KEY LOOKUP
muy frecuente

Puede indicar que falta un índice que cubra la consulta (INCLUDE).



SORT costoso

Puede evitarse con índices que ya entreguen los datos ordenados.



ESTIMACIONES vs. REALES

Si la diferencia de filas estimadas vs reales es muy grande, las estadísticas están desactualizadas.



CONVERT IMPLICIT
en predicados

Puede impedir el uso de índices y provocar scans.



PARALLELISM
(coste y warning)

Puede generar sobrecarga si el plan usa demasiados hilos.



SPOOLS (Eager/Lazy)
frecuentes

Pueden indicar consultas complejas innecesarias.



Revisa siempre las propiedades de cada operador (F4) y el plan XML para un análisis profundo.

5. EJEMPLO PRÁCTICO DE OPTIMIZACIÓN EN GradosTitulos

CONSULTA PROBLEMÁTICA

```
SELECT t.IdTramite, p.Nombres, p.Apellidos, t.FechaRegistro
FROM Tramite.Tramite t
JOIN Academico.Persona p ON t.IdPersona = p.IdPersona
WHERE t.IdEstadoTramite = 2
AND ISNULL(t.FechaRegistro, GETDATE()) BETWEEN '2024-01-01' AND '2024-12-31';
```

- ❌ Uso de ISNULL en columna (evita índices).
- ❌ Falta índice para el filtro por IdEstadoTramite y FechaRegistro.

PLAN PROBLEMÁTICO (RESUMEN)



Diagnóstico: Scan costoso + Key Lookup alto = **Ineficiente**

CONSULTA OPTIMIZADA

```
SELECT t.IdTramite, p.Nombres, p.Apellidos, t.FechaRegistro
FROM Tramite.Tramite t
JOIN Academico.Persona p ON t.IdPersona = p.IdPersona
WHERE t.IdEstadoTramite = 2
AND t.FechaRegistro >= '2024-01-01'
AND t.FechaRegistro < '2025-01-01';
```

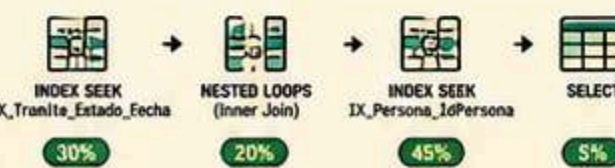
- ✅ Se eliminó ISNULL en el predicado.
- ✅ Se usa rango de fecha SARGable.

ÍNDICES RECOMENDADOS

```
-- Para la tabla Tramite.Tramite
CREATE NONCLUSTERED INDEX IX_Tramite_Estado_Fecha
ON Tramite.Tramite (IdEstadoTramite, FechaRegistro)
INCLUDE (IdPersona, IdTramite);

-- Para la tabla Persona (si no existe)
CREATE INDEX IX_Persona_IdPersona
ON Academico.Persona (IdPersona);
```

PLAN OPTIMIZADO (RESUMEN)



Resultado: Menor costo, menos lecturas, **mejor rendimiento**

BENEFICIOS OBTENIDOS

- ✅ Menos lecturas de disco.
- ✅ Eliminación de Key Lookups innecesarios.
- ✅ Reducción de tiempo de CPU y duración.
- ✅ Mejor uso de índices.
- ✅ Consultas más rápidas para los procesos de Grados y Titulos.



6. DIAGNÓSTICO RÁPIDO CON SENTENCIAS T-SQL

VER PLAN ESTIMADO

```
SET SHOWPLAN_XML ON;
GO
-- Tu consulta aquí
GO
SET SHOWPLAN_XML OFF;
```



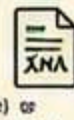
VER PLAN REAL + ESTADÍSTICAS

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO
-- Tu consulta aquí
GO
SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
```



CAPTURAR PLAN EN XML

```
SELECT qp.query_plan, qs.total_logical_reads,
       qs.execution_count, qt.test
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
ORDER BY qs.total_logical_reads DESC;
```



7. BUENAS PRÁCTICAS

- 🔄 Mantén estadísticas actualizadas (UPDATE STATISTICS o AUTO_UPDATE_STATISTICS).
- ✅ Crea índices adecuados y evita índices innecesarios.
- ✅ Evita funciones en columnas dentro de WHERE.
- 🔍 Revisa planes regularmente de consultas críticas (trámites, pagos, sustentaciones, diplomas).
- 🧪 Prueba cambios en ambiente de desarrollo antes de producción.



REGLA DE ORO

UN BUEN PLAN DE EJECUCIÓN =
CONSULTAS RÁPIDAS, RECURSOS OPTIMIZADOS Y USUARIOS SATISFECHOS



CURSO:
BASE DE DATOS

OPTIMIZACIÓN DE CONSULTAS T-SQL

APLICADO A LA BASE DE DATOS: **GradosTitulos**

OBJETIVO: Escribir consultas T-SQL eficientes sobre la base de datos **GradosTitulos** reduciendo lecturas, CPU y tiempo de ejecución, aprovechando índices y buenas prácticas de diseño de consultas.

1. ¿QUÉ ES?



La optimización de consultas consiste en escribir T-SQL eficiente para que SQL Server pueda encontrar y devolver los datos correctos usando la menor cantidad posible de recursos (CPU, memoria, E/S de disco).



Consultas rápidas = Usuarios satisfechos
= Procesos de Grados y Títulos eficientes.

2. EL PELIGRO DE LAS CONSULTAS "NO SARGABLES" (SARGABLE)

Una condición SARGable (Search ARGument ABLE) permite que el motor use el índice.



PRÁCTICA INEFICIENTE (ROMPE EL ÍNDICE)

Aplicar funciones o cálculos sobre la columna impide usar el índice.

```
-- Ejemplo: Buscar trámites por fecha de registro
SELECT IdTramite, FechaRegistro, IdEstadoTramite
FROM Tramite.Tramite
WHERE CONVERT(date, FechaRegistro) = '2024-12-01';
```



SQL Server no puede usar el índice en FechaRegistro.



Provoca SCAN de tabla o índice.



Más lecturas, más CPU, más tiempo.



CÓDIGO OPTIMIZADO (SARGABLE)

Usar rangos sobre la columna, permite usar el índice.

```
SELECT IdTramite, FechaRegistro, IdEstadoTramite
FROM Tramite.Tramite
WHERE FechaRegistro >= '2024-12-01'
AND FechaRegistro < '2024-12-02';
```



SQL Server usa el índice en FechaRegistro.



Menos lecturas, menos CPU, ejecución rápida.



BUENA PRÁCTICA



Evita funciones sobre columnas en WHERE.



Evita el uso innecesario de CONVERT, CAST, LEFT(), RIGHT(), YEAR(), etc.



Usa rangos y columnas indexadas.

3. CONVERSIONES IMPLÍCITAS (TIPOS DE DATOS CRUZADOS)

Cuando se comparan tipos de datos diferentes, SQL Server convierte uno de ellos, rompiendo el uso del índice.



INEFICIENTE

```
SELECT * FROM Academico.Persona
WHERE DNI = 12345678; -- DNI es VARCHAR(8)
```



OPTIMIZADO

```
SELECT * FROM Academico.Persona
WHERE DNI = '12345678';
```



Consejo: Asegúrate de usar el mismo tipo de dato en las comparaciones.

4. OPERADORES OR vs. UNION ALL

El OR suele provocar SCAN. UNION ALL permite usar índices.



OR (INEFICIENTE)

```
SELECT *
FROM Tramite.Tramite
WHERE IdEstadoTramite = 2
OR IdEstadoTramite = 3;
```



Puede causar scan del índice o de la tabla.



Menos eficiente.



UNION ALL (MEJOR)

```
SELECT * FROM Tramite.Tramite
WHERE IdEstadoTramite = 2
UNION ALL
SELECT * FROM Tramite.Tramite
WHERE IdEstadoTramite = 3;
```



Cada consulta usa ese el índice.



Mejor rendimiento.



Usa UNION ALL cuando las condiciones son independientes y no hay duplicados.
Usa UNION (sin ALL) solo si necesitas eliminar duplicados.

5. EXISTENCIA DE DATOS IN / EXISTS vs. LEFT JOIN

Para validar existencia, EXISTS es más eficiente.



LEFT JOIN (INEFICIENTE)

```
SELECT p.IdPersona, p.Nombres
FROM Academico.Persona p
LEFT JOIN Tramite.Tramite t ON t.IdPersona = p.IdPersona
WHERE t.IdTramite IS NOT NULL;
```



EXISTS (MEJOR)

```
SELECT p.IdPersona, p.Nombres
FROM Academico.Persona p
WHERE EXISTS (
    SELECT 1
    FROM Tramite.Tramite t
    WHERE t.IdPersona = p.IdPersona
);
```



EXISTS se detiene al encontrar el primer registro coincidente.

Menos lecturas, mejor rendimiento.

6. MEDICIÓN CIENTÍFICA DEL RENDIMIENTO

Mide siempre antes y después de optimizar.

A. ESTADÍSTICAS DE IO Y TIEMPO

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
```

-- Tu consulta aquí

GO

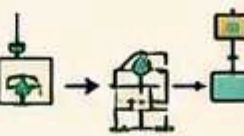
```
SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
```



B. PLAN DE EJECUCIÓN (ESTIMADO O REAL)

Permite ver operadores, costos y uso de índices.

-- Plan estimado: Ctrl + L
-- Plan real: Ctrl + M



C. CONSULTA DE DMVS

Para análisis detallado.

```
SELECT TOP 20
    qs.total_elapsed_time / 1000 AS tiempo_ms,
    qs.execution_count,
    qs.total_logical_reads AS lecturas_logicas,
    SUBSTRING(st.text, (qs.statement_start_offset/2)+1,
        (64*statement_and_offset
        WHEN -1 THEN DATALENGTH(st.text)
        ELSE qs.statement_end_offset
        END - qs.statement_start_offset)/2)+1) AS consulta
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
ORDER BY qs.total_elapsed_time DESC;
```



8. EJEMPLO PRÁCTICO DE OPTIMIZACIÓN EN GradosTitulos

CONSULTA PROBLEMÁTICA

```
SELECT t.IdTramite, p.Nombres, t.FechaRegistro, e.Descripcion
FROM Tramite.Tramite t
INNER JOIN Academico.Persona p ON p.IdPersona = t.IdPersona
INNER JOIN Tramite.EstadoTramite e ON e.IdEstadoTramite = t.IdEstadoTramite
WHERE CONVERT(date, t.FechaRegistro) = '2024-12-01'
AND (e.IdEstadoTramite = 2 OR e.IdEstadoTramite = 3);
```



No SARGable (CONVERT en la columna).



OR puede provocar scan.



Riesgo de alta lectura y CPU.

CONSULTA OPTIMIZADA

```
SELECT t.IdTramite, p.Nombres, t.FechaRegistro, e.Descripcion
FROM Tramite.Tramite t
INNER JOIN Academico.Persona p ON p.IdPersona = t.IdPersona
INNER JOIN Tramite.EstadoTramite e ON e.IdEstadoTramite = t.IdEstadoTramite
WHERE t.FechaRegistro >= '2024-12-01'
AND t.FechaRegistro < '2024-12-02'
AND e.IdEstadoTramite IN (2, 3);
```



SARGable: usa índice en FechaRegistro.



IN permite mejor plan (o en su defecto, dos consultas con UNION ALL).



Menos lecturas, mejor tiempo.

ÍNDICES RECOMENDADOS

IX_Tramite_FechaRegistro
(FechaRegistro)

IX_Tramite_Estado
(IdEstadoTramite)

IX_Tramite_Persona
(IdPersona)



9. CHECKLIST RÁPIDO DE RENDIMIENTO

- ✓ ¿La consulta usa índices? (Verificar en el plan)
- ✓ ¿Es SARGable?
- ✓ ¿Se evitan conversiones implícitas?
- ✓ ¿Se evita SELECT *?
- ✓ ¿Los filtros están en WHERE y son selectivos?
- ✓ ¿Se usan EXISTS / IN correctamente?
- ✓ ¿Se midió con STATISTICS IO y TIME?
- ✓ ¿Se probó nuevamente después de optimizar?



REGLA DE ORO

UN BUEN SQL = MENOS RECURSOS, MÁS VELOCIDAD Y
MEJOR EXPERIENCIA EN LOS PROCESOS DE **GRADOS Y TÍTULOS**.



CURSO:
BASE DE DATOS

CONTROL DE RECURSOS CON RESOURCE GOVERNOR

APLICADO A LA BASE DE DATOS: **GradosTítulos**

OBJETIVO: Aislar y gestionar el consumo de CPU, memoria y E/S en SQL Server para garantizar que los procesos críticos de **GradosTítulos** tengan siempre el rendimiento necesario.

1. ¿QUÉ ES?



Resource Governor es una característica de SQL Server Enterprise que permite administrar y limitar el uso de recursos (CPU, memoria y E/S) por grupos de trabajo (workloads), evitando que una carga pesada degrade el rendimiento de otras cargas críticas.



En **GradosTítulos** garantiza que procesos críticos (trámites, sustentaciones, pagos, diplomas) no sean afectados por consultas o reportes pesados.

2. COMPONENTES CENTRALES DE RESOURCE GOVERNOR

RESOURCE POOL



Define el límite máximo de recursos (CPU y memoria) que puede usar un grupo de solicitudes.

WORKLOAD GROUP



Asocia una o más pools. Define prioridad (IMPORTANCE) y opciones adicionales.

CLASSIFIER FUNCTION



Función T-SQL que evalúa cada sesión y la clasifica en un workload group.

CLASSIFIER MAP



Asignación de la función clasificadora al Resource Governor.

EXTERNAL RESOURCE POOL



Permite gobernar I/O a nivel de discos o archivos (E/S).

3. IMPLEMENTACIÓN PASO A PASO (CASO PRÁCTICO: GradosTítulos)

1. CREAR RESOURCE POOLS

Definimos los límites de recursos.

```
CREATE RESOURCE POOL [RP_Criticos]
WITH ( MAX_CPU_PERCENT = 60,
      MAX_MEMORY_PERCENT = 60 );
```

```
CREATE RESOURCE POOL [RP_Reportes]
WITH ( MAX_CPU_PERCENT = 20,
      MAX_MEMORY_PERCENT = 20 );
```

```
CREATE RESOURCE POOL [RP_Otros]
WITH ( MAX_CPU_PERCENT = 20,
      MAX_MEMORY_PERCENT = 20 );
```



2. CREAR WORKLOAD GROUPS

Asociamos los pools y definimos prioridad.

```
CREATE WORKLOAD GROUP [WG_Criticos]
USING [RP_Criticos]
WITH ( IMPORTANCE = HIGH );
```

```
CREATE WORKLOAD GROUP [WG_Reportes]
USING [RP_Reportes]
WITH ( IMPORTANCE = LOW );
```

```
CREATE WORKLOAD GROUP [WG_Otros]
USING [RP_Otros]
WITH ( IMPORTANCE = LOW );
```



4. CREAR CLASSIFIER MAP Y HABILITAR RESOURCE GOVERNOR

Mapeamos la función y habilitamos.

```
CREATE CLASSIFIER MAP [CM_GradosTítulos]
FOR SYSTEM_USER
EXECUTE dbo.fnRG_GradosTítulos;
GO
```

```
ALTER RESOURCE GOVERNOR
WITH ( CLASSIFIER_FUNCTION = dbo.fnRG_GradosTítulos,
      CLASSIFIER_TYPE = 'USER' );
ALTER RESOURCE GOVERNOR RECONFIGURE;
```



5. VALIDAR CONFIGURACIÓN

Verificamos que esté habilitado.

```
SELECT name, is_enabled
FROM sys.resource_governor_configuration;

SELECT pool_id, name, max_cpu_percent,
       max_memory_percent
FROM sys.resource_governor_resource_pools;

SELECT group_id, name, pool_id, importance
FROM sys.resource_governor_workload_groups;
```

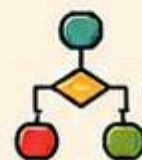


3. CREAR FUNCIÓN CLASIFICADORA

Clasificamos las sesiones según la aplicación.

```
CREATE FUNCTION dbo.fnRG_GradosTítulos()
RETURNS SYSNAME
WITH SCHEMABINDING
AS
BEGIN
    DECLARE @app NVARCHAR(128);
    SELECT @app = APP_NAME();

    -- Aplicaciones definidas en la conexión
    IF @app LIKE 'SISTEMA_GRADOS_TITULOS%'
        RETURN 'WG_Criticos';
    ELSE IF @app LIKE 'REPORTES_GRADOS_TITULOS%'
        RETURN 'WG_Reportes';
    ELSE
        RETURN 'WG_Otros';
END
GO
GO
```



6. ASIGNAR NOMBRE DE APLICACIÓN EN LAS CONEXIONES (IMPORTANTE)

Ejemplos de cadenas de conexión.

- Sistema críticas:
APP= SISTEMA_GRADOS_TITULOS
- Reportes:
APP= REPORTES_GRADOS_TITULOS
- Otras:
APP= OTRAS_APLICACIONES



4. MONITOREO Y VALIDACIÓN DEL AISLAMIENTO

Consultas DMV para monitorear en tiempo real.



USO DE CPU POR POOL

```
SELECT pool_id, name, cpu_usage_percent
FROM sys.dm_resource_governor_resource_pools;
```



USO DE MEMORIA POR POOL

```
SELECT pool_id, name, physical_memory_in_use_kb
FROM sys.dm_resource_governor_resource_pools;
```



SESIONES Y GRUPOS DE TRABAJO

```
SELECT session_id, login_name, program_name,
       rg.pool_id, rg.group_id, status
FROM sys.dm_exec_sessions;
```



ESTADÍSTICAS DETALLADAS

```
SELECT * FROM sys.dm_resource_governor_stats;
```

Interpretación rápida

- Pool con uso bajo: Recursos disponibles.
- Pool cerca del límite: Ajustar o redistribuir.
- Pool al 100%: Puede impactar a otras cargas.



5. EJEMPLO DE ESCENARIO EN GradosTítulos

Procesos Críticos (WG_Criticos)

- Registro de Trámites
- Pagos y validaciones
- Sustentaciones
- Emisión de diplomas



Procesos No Críticos (WG_Reportes)

- Reportes de gestión
- Consultas analíticas pesadas
- Exportaciones de datos
- Consultas ad-hoc



Resultado: Los procesos críticos siempre tendrán prioridad y no serán afectados por los reportes o consultas pesadas.

6. ADVERTENCIAS CRÍTICAS DE IMPLEMENTACIÓN



NO SOBRESTIMAR LOS LÍMITES

Valores muy bajos pueden causar esperas y degradar el rendimiento en general.



PROBAR ANTES EN PRUEBAS

Validar en un entorno de pruebas antes de aplicar en producción.



MONITOREO CONTINUO

El comportamiento de las cargas cambia. Ajusta periódicamente los límites y prioridades.



Resource Governor no reemplaza el tuning de consultas ni los índices. Es una capa de protección y administración de recursos.

7. BUENAS PRÁCTICAS



Definir pools según prioridad del negocio, no por número de usuarios.



Usar APP_NAME() para clasificar correctamente las sesiones.



Monitorear regularmente con DMVs y ajustar si es necesario.



Proteger siempre las cargas críticas de **GradosTítulos**.



Usar External Resource Pool para gobernar I/O cuando sea necesario.



REGLA DE ORO

RECURSOS CONTROLADOS = PROCESOS CRÍTICOS PROTEGIDOS
USUARIOS SATISFECHOS = MEJOR EXPERIENCIA EN **Grados y Títulos**



CURSO:
BASE DE DATOS